

SAiDL Induction Assignment 2026

by AB

Contents

1	Introduction	2
1.1	Assignment	2
1.2	Assignment-1	2
1.3	Assignment-2	2
2	Core ML Assignment	3
2.1	Introduction	3
2.2	Baseline Attention	4
2.2.1	Learnings from the paper	4
2.2.2	Implementation Assumptions	4
2.2.3	Implementation Learnings	4
2.3	Different Attention Mechanism	5
2.3.1	Learnings from the paper	5
2.3.2	Implementation Assumptions	5
2.3.3	Implementation Learnings	5
2.4	Experiment Analysis	6
2.4.1	Graphs	6
2.4.2	Inference	9
2.4.3	Limitations	10
2.5	Different Positional Encoding Implementation	11
2.6	Convolution Attention Implementation	12
3	Diffusion	12
3.1	Positional Encoding Ablation	13
3.2	Throughput Analysis	14
3.3	Downstream Perplexity	15

1 Introduction

1.1 Assignment

- This Report attempts to tackle the Core-ML and Diffusion problem statement from the SAiDL Induction 2026 Assignment
- All evaluation is done on a NVIDIA GeForce RTX 3050 6GB Laptop GPU, so all the results pertaining to GPU analysis, throughput etc are in accordance with aforementioned device
- GitHub Repo- https://github.com/Ad-B-R/SAiDL_Induction_Assignment

1.2 Assignment-1

- Core-ML Assignment tackles the empirical study of different attention variants and their performance.
- For this assignment, Sliding Window, Grouped Query Attention and SoftMax-Free Attention were used alongside the baseline Attention model
- For Positional Embeddings- Standard sine-cosine absolute encoding, ALiBi, RoPE and RPE were used
- Lastly, the Convolution + Attention model uses (insert model) based on the empirical results from the 2 and 3 subsection of the assignment

1.3 Assignment-2

- Core-ML Assignment tackles the empirical study of different attention variants and their performance.
- For this assignment, Sliding Window, Grouped Query Attention and SoftMax-Free Attention were used alongside the baseline Attention model
- For Positional Embeddings- Standard sine-cosine absolute encoding, ALiBi, RoPE and RPE were used
- Lastly, the Convolution + Attention model uses (insert model) based on the empirical results from the 2 and 3 subsection of the assignment

2 Core ML Assignment

2.1 Introduction

- As was discussed earlier, this assignment is the empirical study of different attention variants and their performance, incorporating ablation study of the context window etc.

2.2 Baseline Attention

2.2.1 Learnings from the paper

- Architecture behind transformer (the FFN, use of Residual connection in conjunction with the Add+Norm Layer) specifically the Decoder-Part
- Casual masking, which retains the auto-regressive property that is needed to make the architecture for wikitext-2 dataset, whose implementation mimics GPT's (Generative Pre-Trained Transformer) work
- It also described the absolute sin-cosine positional encoding (which we will call "standard" encoding from now on)
- The attention formula itself which is the basis of the "attention" block present in the architecture (and would be the point of introduction of RoPE and ALiBi)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Need of two separate embeddings- Positional Embedding and Word Embedding, whereby one dictates the Position of individual sequence's order, crucial for NLP tasks; the other one to actually give words meaning in a vector form

2.2.2 Implementation Assumptions

-

2.2.3 Implementation Learnings

2.3 Different Attention Mechanism

2.3.1 Learnings from the paper

2.3.2 Implementation Assumptions

2.3.3 Implementation Learnings

2.4 Experiment Analysis

2.4.1 Graphs

- All the following graphs are evaluated on Absolute Standard Positional encoding, ranging from context length $L_{train} = [256, 512, 768, 1024]$
- The Attention Mechanism tested are- Grouped Query Attention (GQA), SoftMax-Free Attention (ReLU-Attention) and Sliding Window Attention

Analysis for $L_{train} = 256$

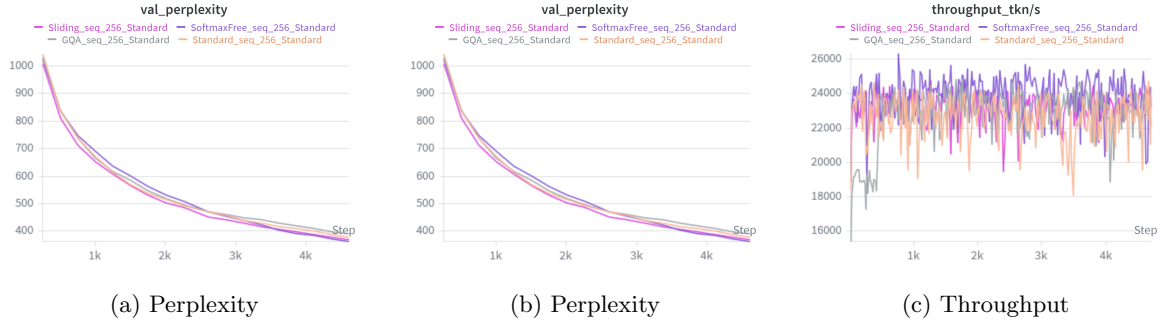


Figure 1: Results from $L_{train} = 256$ indicate SoftMax-Free has the highest throughput meanwhile having the lowest perplexity

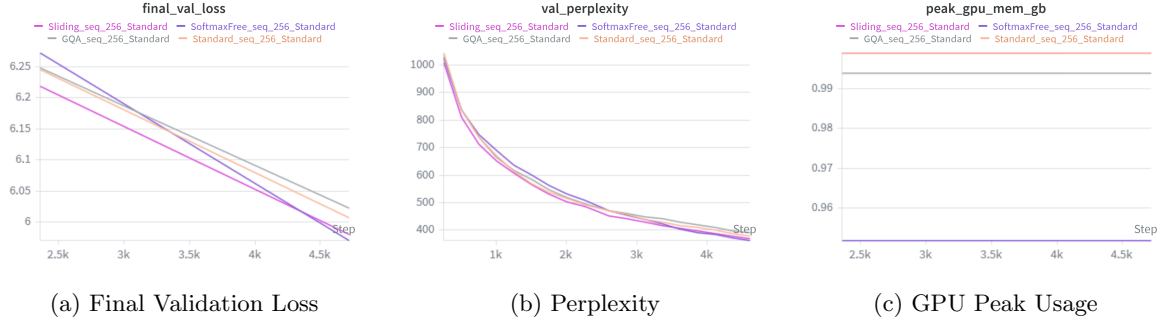


Figure 2: Results from $L_{train} = 256$ indicate SoftMax-Free has the least GPU mem usage meanwhile having the highest negative final validation loss slope

- It is imperative to note that while SoftMax-Free performed the best in terms of validation loss and perplexity (all other graphs relevant to this are attached in Subsection 2.7 which might show a slight variance but this is the trend for the major important graphs), Sliding Window Consistently performed the 2nd best for $L_{train} = 256$, a fact which will become relevant for the other sequence length further. It can also be noted the GQA had the highest perplexity, val loss and throughput, while it had the 3rd highest GPU peak Usage, hence making it the worst performer in this bunch.

Analysis for $L_{train} = 512$

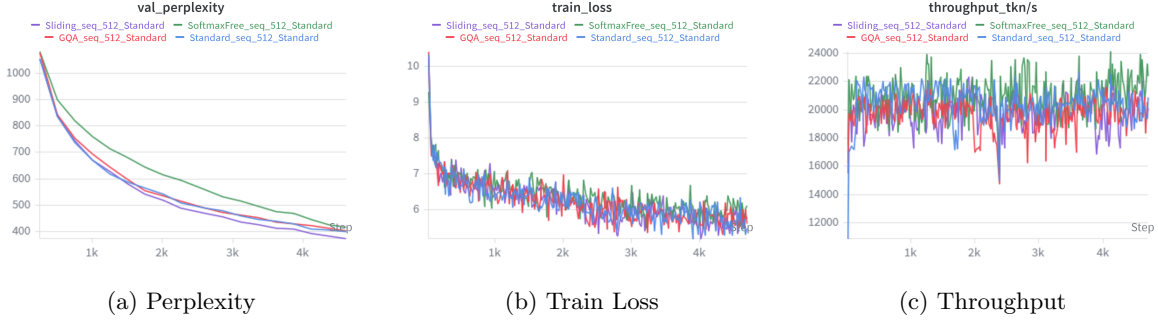


Figure 3: Results from $L_{train} = 512$ indicate SoftMax-Free has the highest throughput while having the highest perplexity meanwhile Sliding Window has the lowest perplexity

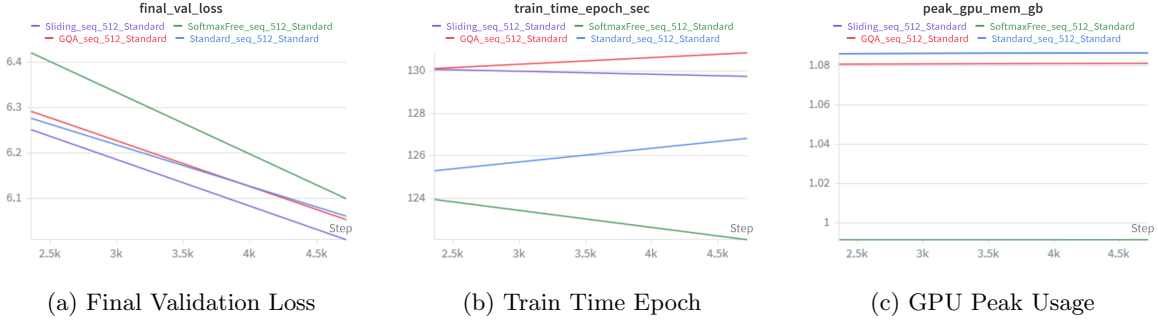


Figure 4: Results from $L_{train} = 512$ indicate SoftMax-Free has the least GPU mem usage meanwhile having the highest final validation loss (and slope too) while Sliding Window has the lowest final validation loss while having same peak mem usage as Standard

- It is imperative to note that SoftMax-Free performed the best in terms of peak gpu memory usage and the throughput, it performed drastically worse on perplexity and val loss (having the highest by a long margin compared to the others). This would contribute to some interesting inferences as will be discussed later.
- Sliding Window consistently performed the best on perplexity/loss front by having the least perplexity/loss by far (Although a high peak gpu memory usage and the least throughput).
- Standard and GQA showed identical trends in all metrics
- For $L_{train} = 768, 1024$, a similar trend was observed as $L_{train} = 512$

Analysis for $L_{train} = 768$

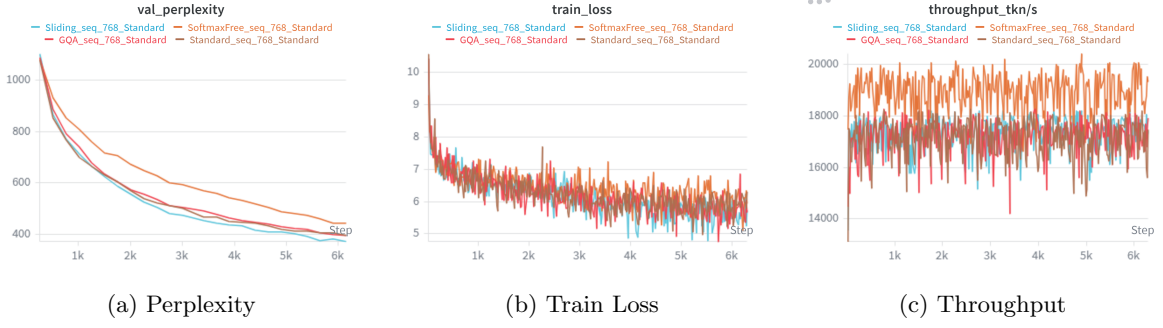


Figure 5: Results from $L_{train} = 768$ indicate SoftMax-Free has the highest throughput while having the highest perplexity meanwhile Sliding Window has the lowest perplexity

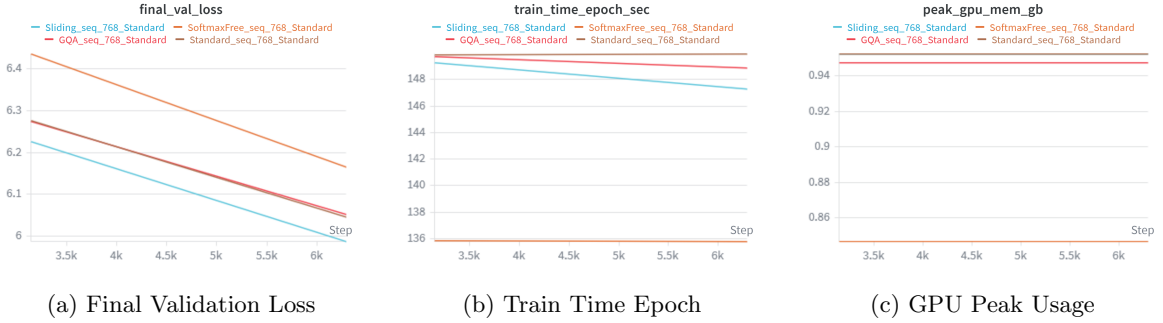


Figure 6: Results from $L_{train} = 768$ indicate SoftMax-Free has the least GPU mem usage meanwhile having the highest final validation loss (and slope too) while Sliding Window has the lowest final validation loss while having same peak mem usage as Standard

Analysis for $L_{train} = 1024$

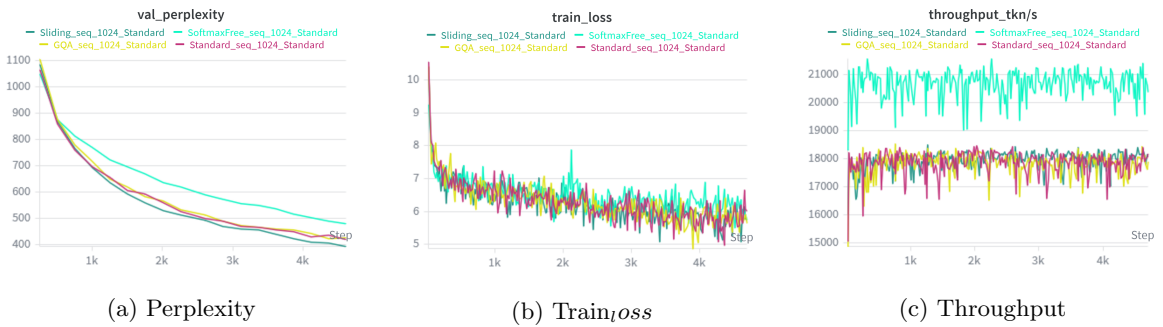


Figure 7: Results from $L_{train} = 1024$ indicate SoftMax-Free has the highest throughput while having the highest perplexity meanwhile Sliding Window has the lowest perplexity

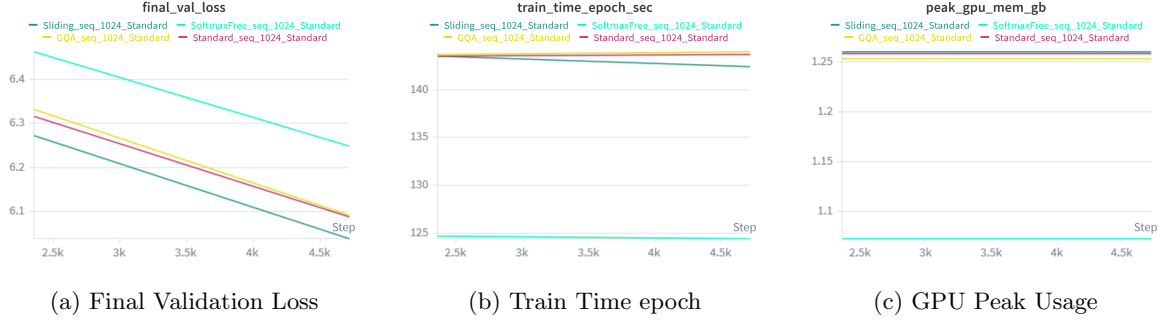


Figure 8: Results from $L_{train} = 1024$ indicate SoftMax-Free has the least GPU mem usage meanwhile having the highest final validation loss (and slope too) while Sliding Window has the lowest final validation loss while having same peak mem usage as Standard

2.4.2 Inference

2.4.3 Limitations

- Test
- Sliding Window isn't faithful to the paper since the implementation doesn't change the CUDA Kernel itself, hence the peak memory usage won't be accurate representation of the actual "intended" performance

2.5 Different Positional Encoding Implementation

2.6 Convolution Attention Implementation

3 Diffusion

3.1 Positional Encoding Ablation

3.2 Throughput Analysis

3.3 Downstream Perplexity